```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# --- 1. Load the Dataset ---
# As specified in the image: sklearn.datasets
diabetes = datasets.load_diabetes()
X = diabetes.data
y = diabetes.target

# Feature names mapping (Age, Sex, BMI, BP, s1, s2, s3, s4, s5, s6)
feature_names = diabetes.feature_names

# --- DATA TRANSFORMATION FOR CLASSIFICATION ---
# The standard sklearn diabetes dataset is for regression.
# To generate the Confusion Matrix requested in the image, we classify
# patients into 'High Risk' (1) or 'Low Risk' (0) based on the median progression.
threshold = np.median(y)
y_binary = (y > threshold).astype(int)

print(f"Dataset Loaded. Features: {X.shape[1]}, Samples: {X.shape[0]}")
print("Target converted to binary classes for Classification task.\n")

# --- 2. Data Normalization and Preprocessing ---
# SVM is sensitive to unscaled data, so we must standardize it.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split data into training and testing sets (80% Train, 20% Test)
X_train, X_test, y_train, y_test =
train_test_split(X_scaled, y_binary, test_size=0.2, random_state=42)

# --- 3. SVM Implementation ---
# Methodology from image: "Apply kernel-based transformation using RBF kernel"
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale')

# Train the model
svm_model.fit(X_train, y_train)

# --- 4. Classify Disease Outcomes ---
y_pred = svm_model.predict(X_test)

# --- 5. Evaluation Report ---
print("--- Evaluation Report ---\n")

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}\n")

# Classification Report (Precision, Recall, F1-score)
print("Classification Report:")
```

```
print(classification_report(y_test, y_pred, target_names=
 ['Low Progression', 'High Progression']))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Plotting the Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted Low', 'Predicted High'],
            yticklabels=['Actual Low', 'Actual High'])
plt.title('Confusion Matrix: Diabetes Diagnosis (SVM)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
```

```
Dataset Loaded. Features: 10, Samples: 442
Target converted to binary classes for Classification task.

--- Evaluation Report ---

Accuracy: 0.75

Classification Report:
                  precision    recall  f1-score   support

 Low Progression       0.80      0.73      0.77        49
High Progression       0.70      0.78      0.74        40

        accuracy                           0.75        89
       macro avg       0.75      0.75      0.75        89
    weighted avg       0.76      0.75      0.75        89
```